

Social Housing Tenant Portal

<https://socialhousingtenantportal.live/>

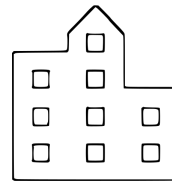
Jack Jarvis

Kacper Zdrada

Mohammad Alhazza

Oliver Tchum

Arav Raja



JAFFERAI

Project Brief

The vision: An AI enhanced tool, developed for JafferAI, to allow tenants to efficiently report maintenance issues to housing associations/landlords, and have the requests forwarded to maintenance teams.

The client:

- **JAFFERAI**
- Startup company comprised of two employees (CEO and technical lead)
- Have liaised with both throughout the project

Stakeholders:

Tenants, Housing Associations/Landlords, Repair Personnel, Councils, Legislators (GDPR)

Motivation

Current process for reporting issues is extremely inefficient



Call centres are often unable to accurately diagnose issues on the phone



Surveyors are often required to visit the property for diagnosis before a maintenance team can visit



Many follow up visits often required



Our system will reduce the number of visits needed by providing maintenance teams with comprehensive information about the issue before they arrive.

User Stories



As a **tenant**, I want to get immediate advice for what my issue could be caused by, and get a repair scheduled without needing inconvenient assessment or multiple follow-up visits.

This saves me a lot of time, and stress, and helps my problem be promptly fixed.



As a **housing association or landlord**, I want to get an accurate diagnosis of the issue a tenant is facing, to eliminate the need of costly assessment and follow-up repair visits.

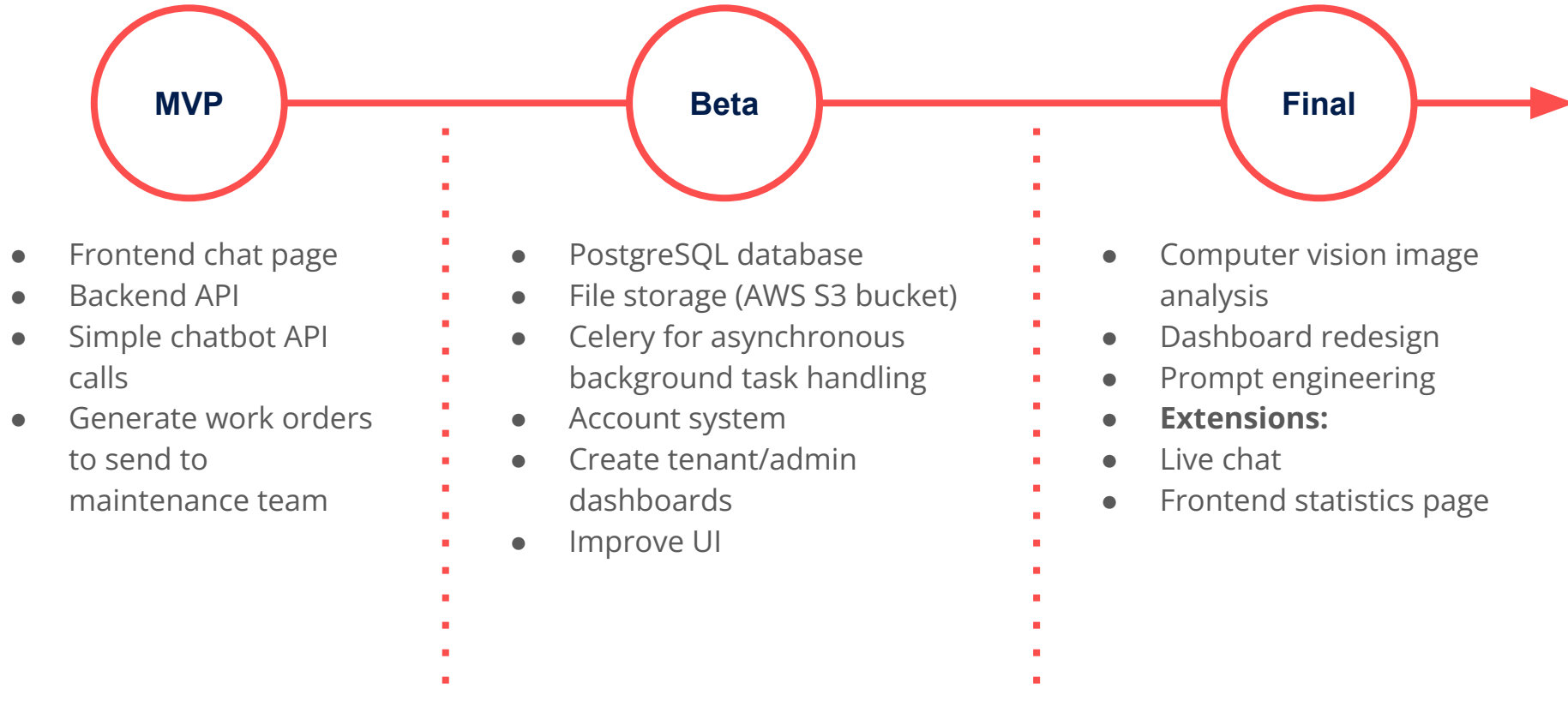
This way, our limited financial resources can be better spent in other areas.



As a **tradesperson** I want to have a comprehensive understanding of the problem before I arrive at the property.

This will allow me to immediately bring the necessary tools and parts, saving time, and minimising the often challenging issue of scheduling multiple appointments with tenants.

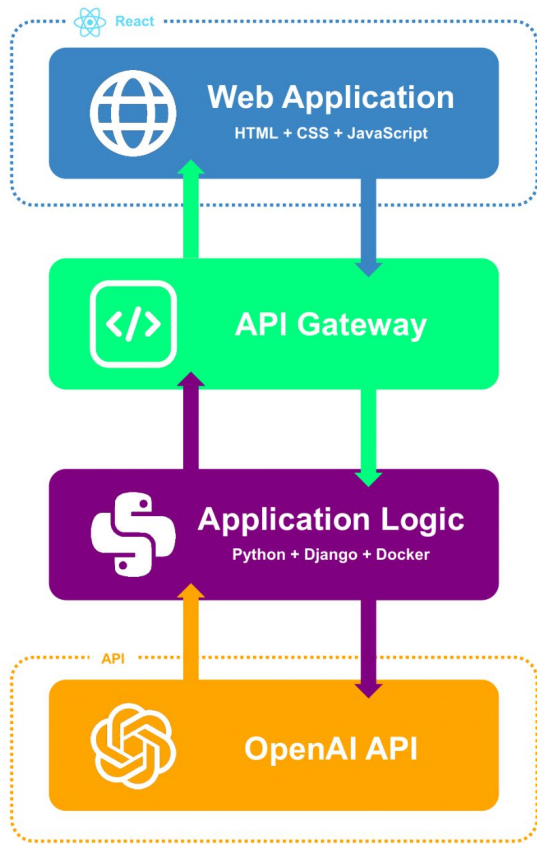
Requirements Timeline



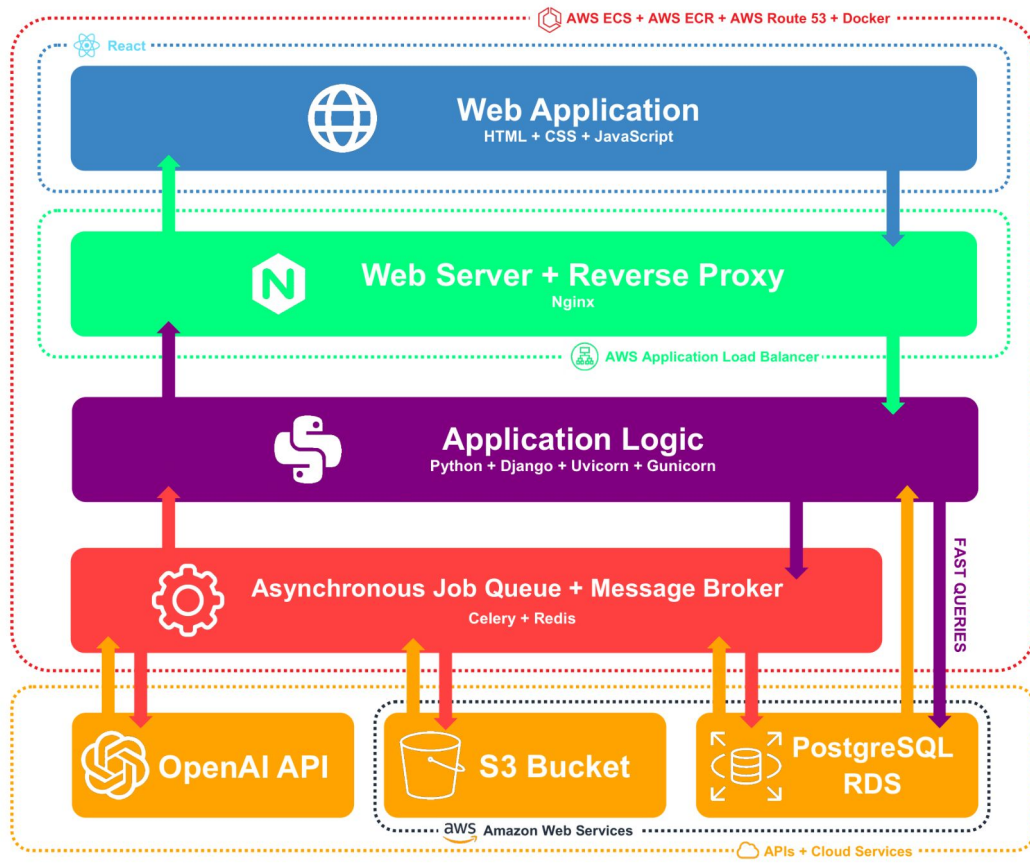
Product Demo

<https://socialhousingtenantportal.live/>

MVP Stack



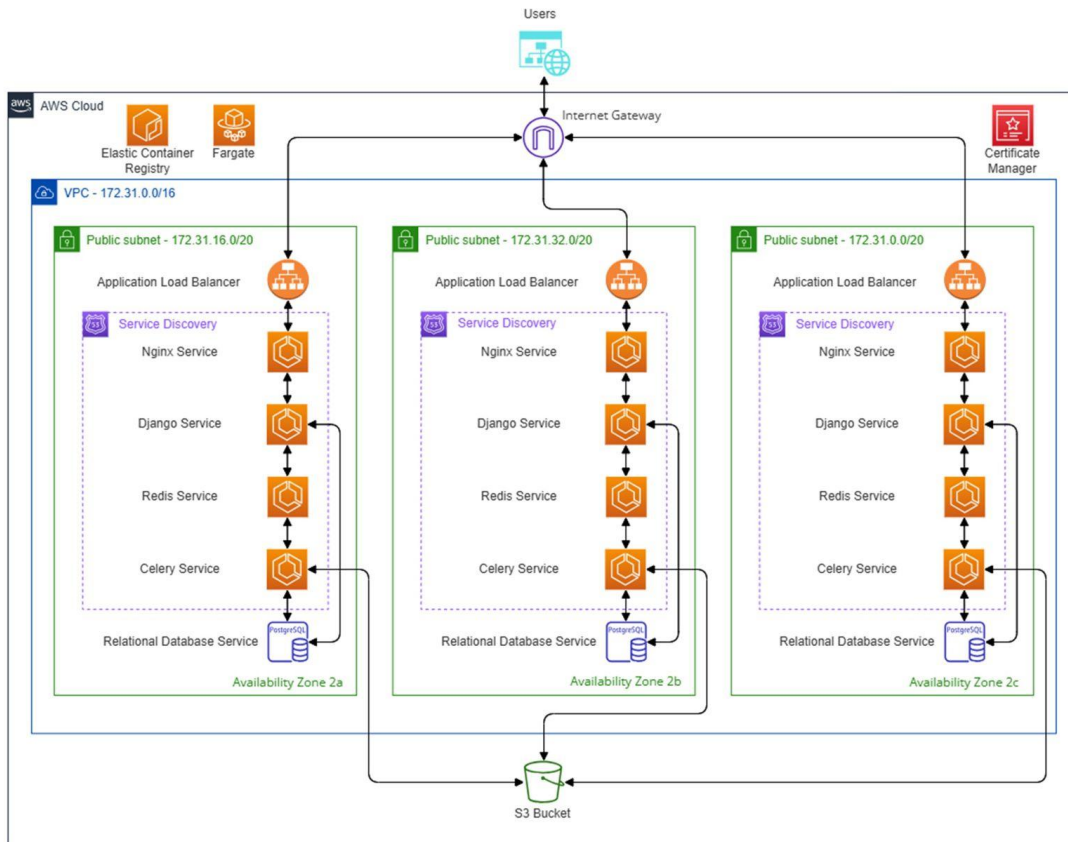
Final Stack



Deployment

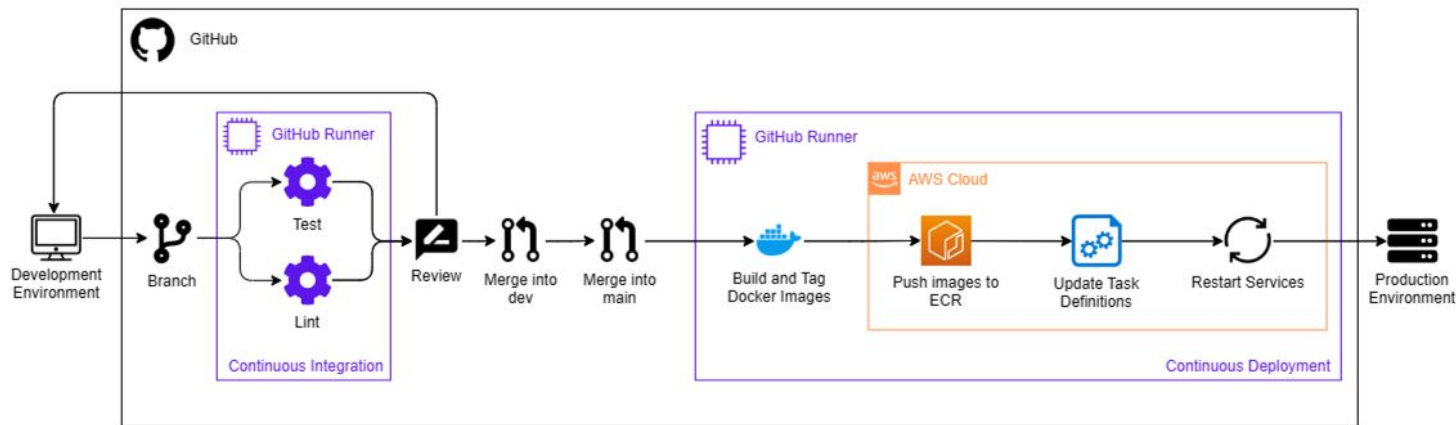
AWS Cloud Architecture

- Elastic Container Service running Docker images
- Elastic Container Registry holding Docker images
- Application Load Balancer that domain is registered to
- Route 53 Service Discovery to allow ECS communication
- PostgreSQL Relational Database Service
- S3 Bucket holding larger files
- Certificate Manager holding SSL certificate



Development Process

- We use GitHub Actions for automated Continuous Integration and Continuous Deployment
- CI runs our test suites and linting on pushes and merges
- CD runs on merges into main and deploys all changes onto AWS



Code Testing

Coverage report: 93%

Files Functions Classes

coverage.py v7.8.0, created at 2025-04-19 16:47 +0000

File	statements	missing	excluded	coverage
dashboard/views.py	520	18	0	97%
dashboard/tests/test_invalid.py	378	0	0	100%
authentication/views.py	334	75	0	78%
dashboard/tests/test_valid.py	324	0	0	100%
authentication/tests/test_invalid.py	234	0	0	100%
chatbot/open_ai/make_pdf.py	213	50	0	77%
authentication/tests/test_valid.py	188	0	0	100%
chatbot/models.py	149	21	0	86%
live_chat/consumers.py	138	27	0	80%
chatbot/open_ai/api.py	138	44	0	68%
live_chat/tests/test_valid.py	120	0	0	100%
chatbot/views.py	108	21	0	81%

Utilised **Test Driven Development** to ensure all code functions as expected.

This included both **unit** and **integration** tests.

- Testing suite for Django backend - **code coverage: 93%**
- Testing suite for React frontend - **code coverage: 80%**
- Linting for both to ensure code quality is high

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	80.37	74.55	75.63	82.06	
components	78.23	72.96	72.22	79.27	
Chat.js	74.76	68.51	67.12	76.02	...22, 527-529, 535-537, 573, 623-631, 691, 745-755, 824
Home.js	80	87.5	66.66	80	62
Layout.js	91.83	86.84	75	91.83	63, 173, 212-215
Settings.js	77.7	75	80.76	78	...18, 132-133, 234-235, 273, 302, 309-335, 387-391, 550
Tutorial.js	93.75	80.95	87.5	96.66	170
components/authentication	91.8	82.14	94.87	91.71	
AdditionalInfo.js	89.28	67.74	83.33	89.09	53, 93, 127-128, 250-270
GoogleLoginButton.js	100	100	100	100	
GoogleSignupButton.js	100	100	100	100	
Login.js	93.47	90.9	100	93.47	49-50, 80
Signup.js	91.04	90.32	100	90.9	123-124, 141-145
components/dashboard	77.02	74.25	72.41	79.95	
AdminDashboard.js	76.61	71.3	73.13	79.21	...19, 324-341, 347, 372, 444, 446, 472, 489-490, 515-539
Dashboard.js	70.26	79.16	63.63	80	126-142, 148-164, 170-183
StatisticsDashboard.js	74.22	60	69.23	80.23	87, 113, 138, 163, 199-210, 215-217, 241-247
TenantDashboard.js	79.61	81.33	75.6	81.05	44, 51-52, 142-146, 151-165, 196, 264, 281-312
contexts	100	83.33	100	100	
AuthContext.js	100	83.33	100	100	48-53

User Testing / Evaluation

Testing
Day

- Average SUS (System Usability Scale) score of **92.7/100**
 - **13** participants
 - **5-10 minute** test
-

Testing
Day 2

- Average SUS (System Usability Scale) score of **83.3/100**
 - **12** participants
 - **10-15 minute** test
-

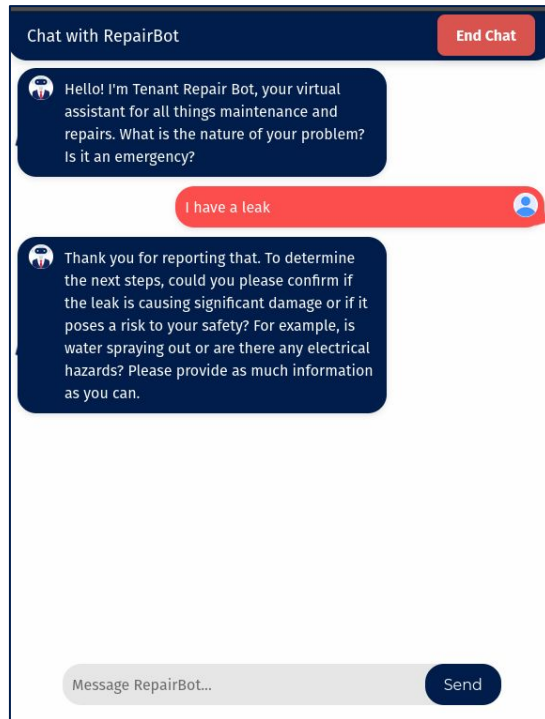
External

- Average SUS (System Usability Scale) score of **94.1/100**
- **12** participants
- **15-20** minute test

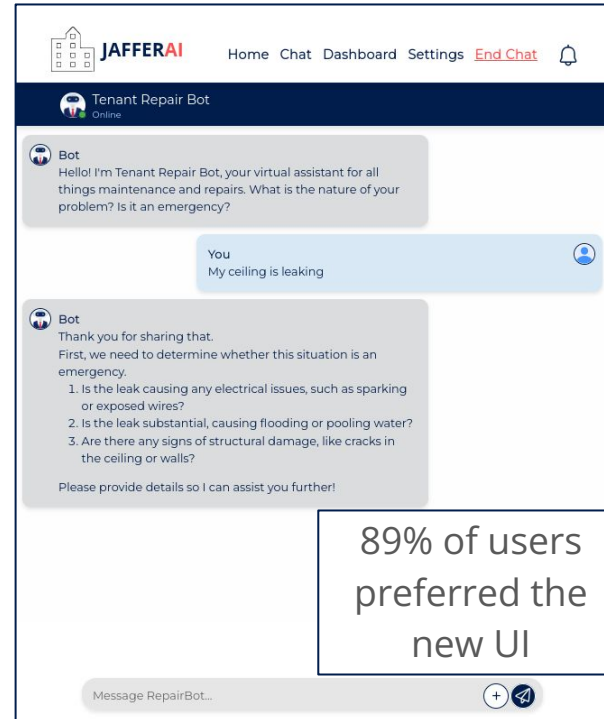
User Testing / Evaluation - Styling

- Users found the colour scheme **too bright**
- UI described as **“childish”**
- Text and background colours also failed the [Web Content Accessibility Guidelines \(WCAG\)](#) requirements for contrast, meaning the text was **difficult to read** for some people.

Before



After



User Testing / Evaluation - Message Box

- Users noted the text box didn't expand when they typed multiple lines in a message
- This made it difficult to type long messages

We updated the message box styling to fix this:

Before



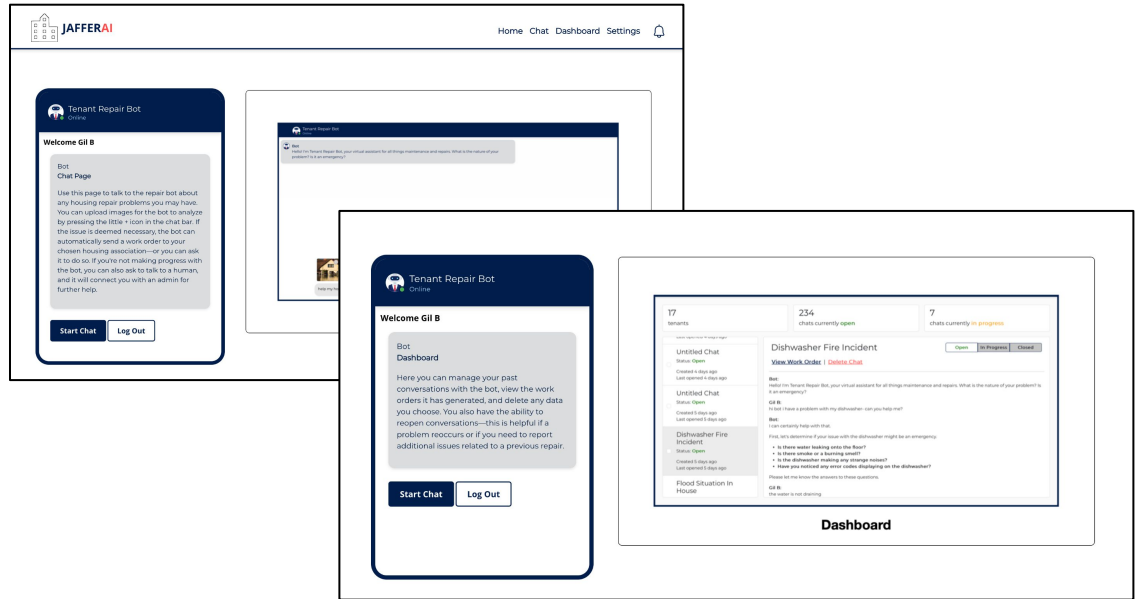
After



User Testing / Evaluation - Tutorial Page

- Users **did not realise** some features were available, such as live-chat and image-analysis.
- This meant without prompting, **users were not interacting with some features**

We added tutorial pages to explain all features of our site in detail.



User Testing / Evaluation - Dashboard Before

Admin Dashboard

Use this page to manage tenants and view their chats and workorders.

Arav Raja

Arav Raja

Arav Raja

Arav Raja

Arav Raja

Arthur Jarvis

Boboy Raja

dede ffefer

Ethan McClain

Gil B

Glob Hob

J J

● Chat 1

● Chat 2

Bot:

Hello! I'm Tenant Repair Bot, your virtual assistant for all things maintenance and repairs. What is the nature of your problem? Is it an emergency?

You:

hi chat can u connect to human

Bot:

It seems you may want to discuss something that is not related to repairs. If you have a specific issue with maintenance or repairs, please let me know, and I'll be happy to assist you! If you still prefer to speak with a human operator, I can help with that as well.

You:

conenct to human

View Work Order

Set Status ▾

Users found the design unintuitive and cumbersome

Status Key

- Open
- In Progress
- Closed

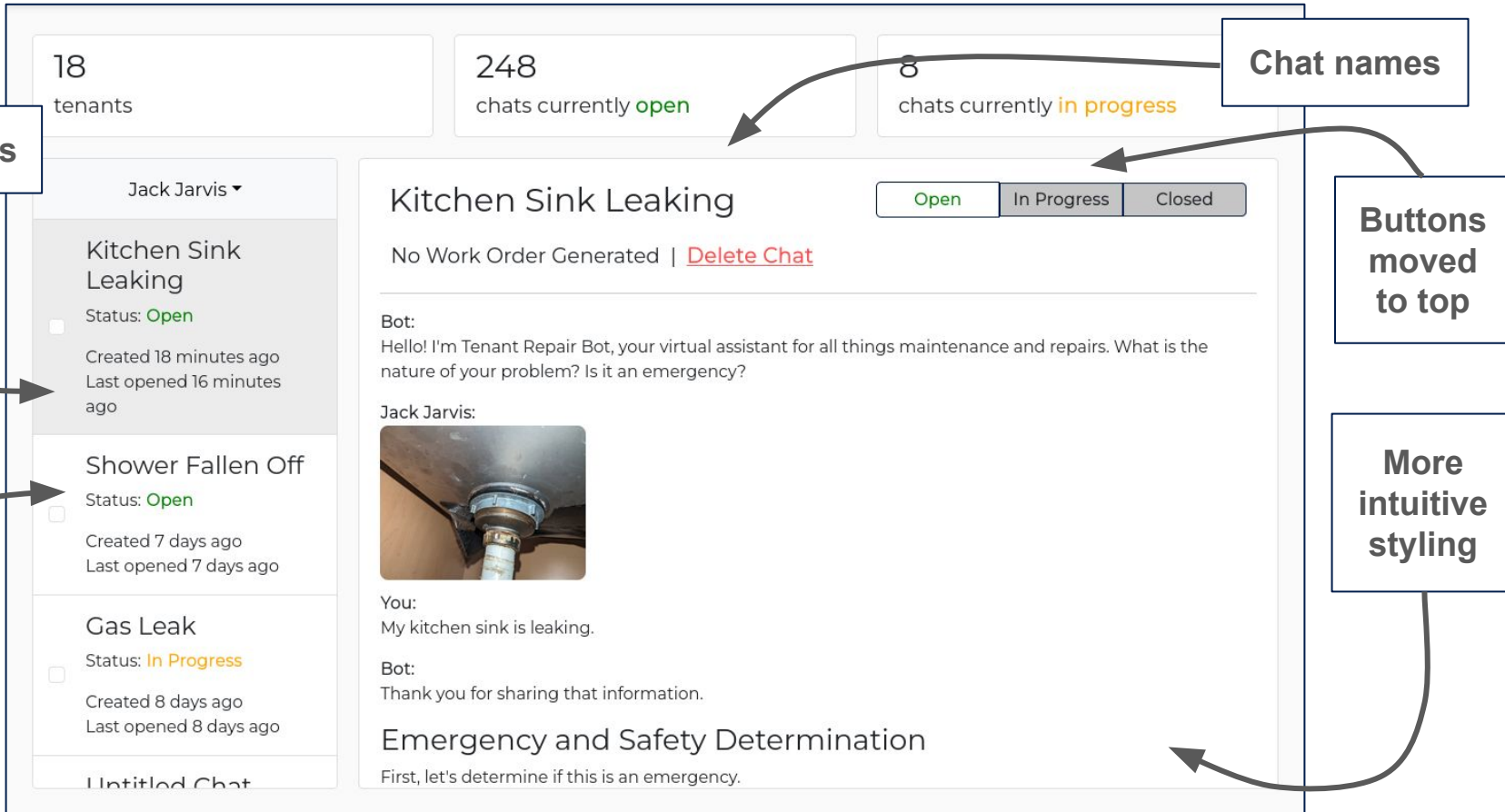
Styling described as childish

Buttons at the bottom - scrolling often needed to reach them

No chat names / timestamps

Chats are sorted arbitrarily

User Testing / Evaluation - Dashboard After



Client feedback

Positives

- ✓ The client was very happy with the product
- ✓ Impressed with dashboard pages
- ✓ Believes addition of live chat is very useful as an alternative user flow

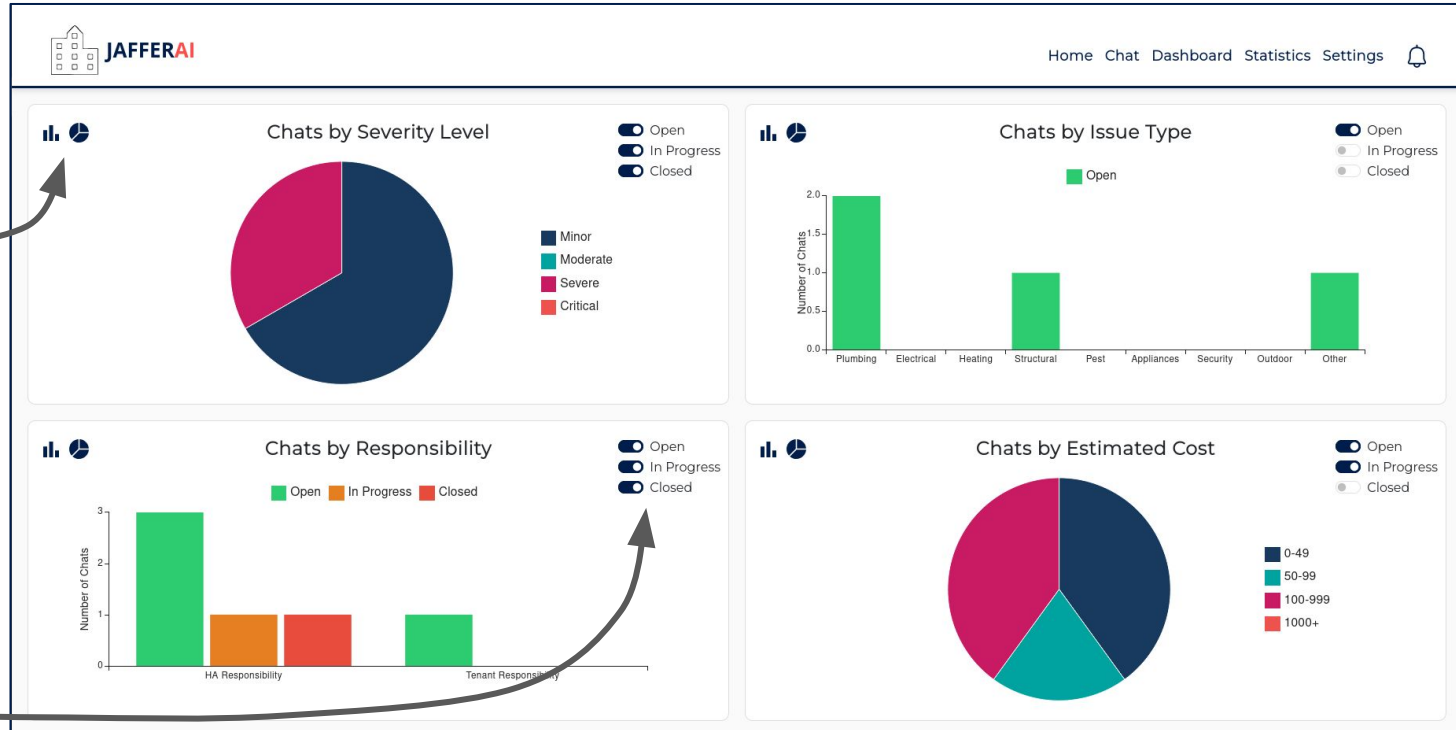
Client feedback

Problems

-  At MVP release, the client too thought the initial chat UI was a bit basic and uneasy on the eyes
-  Raised concerns about the quality of the issue diagnosis provided by the chatbot
-  Client suggested displaying more in depth statistics about chats and tenants on the dashboard page

Client feedback - Statistics Page

We added a statistics page to show more detailed chat information:



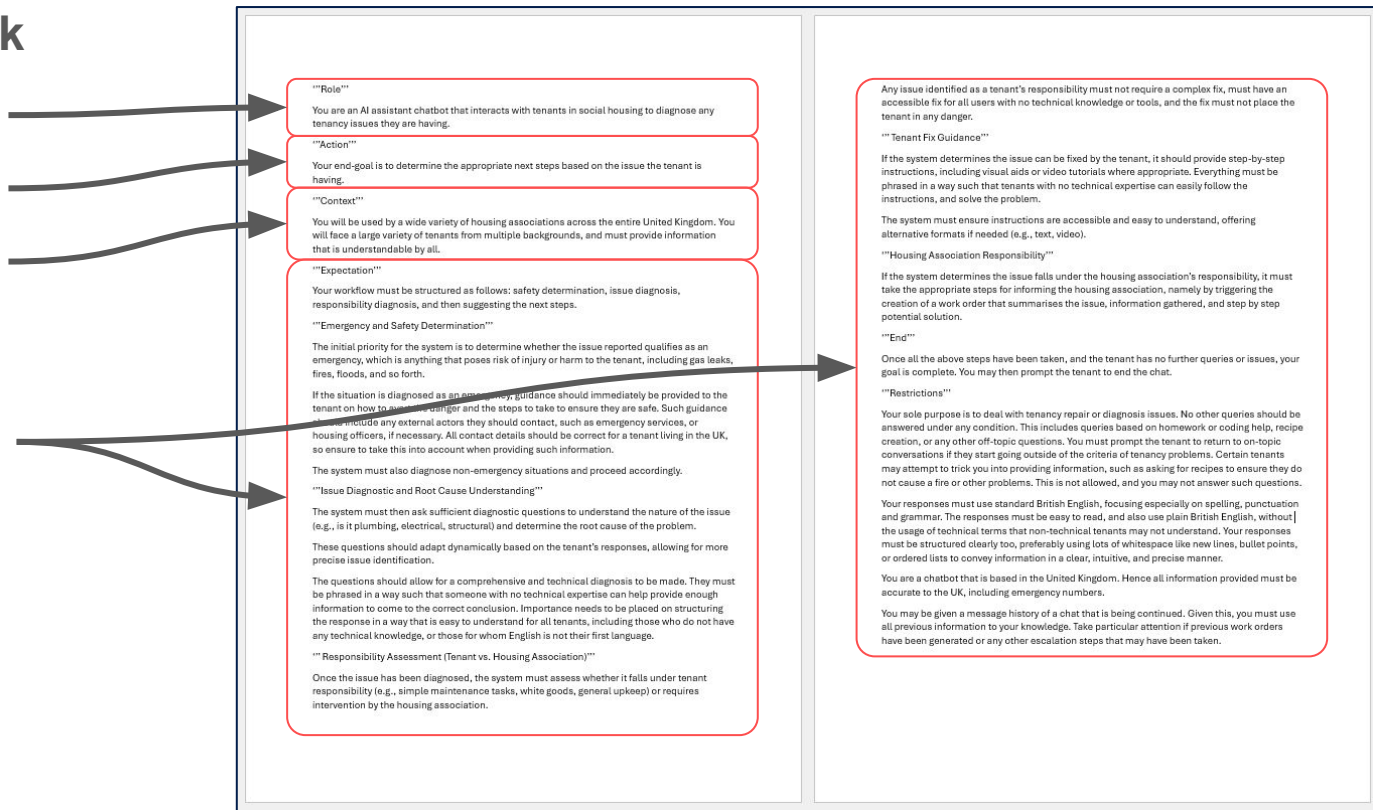
Toggle between pie and bar charts

Filter by chat status

Client/User Feedback - Prompt Engineering

RACE Framework

- Role
- Action
- Context
- Expectation
(with Chain of Thought Reasoning)



Compliance with GDPR

- **Lawfulness, Fairness, and Transparency**
 - There is a lawful basis for the data being collected (the diagnoses of tenant issues), data processing will be fair (the data will only be used for the diagnoses of tenant issues)
- **Purpose Limitation**
 - AI only collects data about repair issues
- **Data Minimisation**
 - AI will finish the chat and stop asking questions after work order generation
- **Accuracy**
 - Data can be updated through the dashboard and settings pages
- **Storage Limitation**
 - Data can be updated/deleted through the dashboard and settings pages
- **Integrity and Confidentiality**
 - TLS used, all data stored securely in AWS RDS and S3 buckets
 - Robust authentication/authorisation system used

Legality

CURRENT LEGAL CONSIDERATIONS

No general statutory regulation for AI within the UK at the moment

However this project may already be encompassed by existing law like:

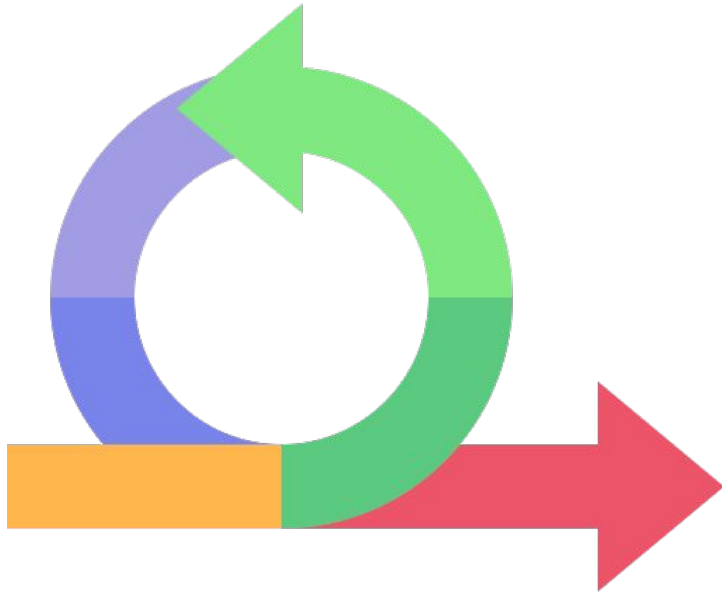
- The Consumer Protection Act 1987
- Equality Act 2010
- Online Safety Act 2023
- GDPR
- Data Protection Act 2018

FUTURE LEGAL CONSIDERATIONS

The project has been evaluated using the following principles proposed by the UK government, and mitigating actions suggested:

- Safety, security, and robustness
- Appropriate transparency and explainability
- Fairness
- Accountability and governance
- Contestability and redress

Agile Development / Project Management



We held **weekly standup meetings** in which:

- Work done as of the last meeting discussed
- Work expected to be done by the next meeting was planned
- The **backlog** and items **to-do** were evaluated
- Any **blocking** backlog tasks were emphasised

Agile Development / Project Management

SPRINTS

We utilised 4 sprints, inline with our development timeline, in order to boost motivation and project development

PAIR PROGRAMMING

Used when individuals were struggling on a bug, was important to maintain project momentum, and keep morale high

COMMUNICATION

Discord was used as the main discussion forum, with 14 message channels and over 13,500 messages sent

Agile Development / Project Management

☐	6 Open ✓ 43 Closed	Author ▾	Label ▾	Projects ▾	Milestones ▾	Reviews ▾	Assignee ▾	Sort ▾
☐	Chat Statistics ✓ backend enhancement PR Priority: High #229 opened 3 days ago by KacperZdrada • Review required 14 tasks Final Release							4
☐	Increased backend test coverage ✓ backend PR Priority: Medium testing #226 opened 3 days ago by jack-jarvis12 • Changes requested 10 of 14 tasks Final Release				4			7
☐	Welcome overhaul ✗ enhancement frontend PR Priority: Low #221 opened 5 days ago by AravRaja • Draft 14 tasks							
☐	Enhancements and bug fixes ✓ backend bug enhancement PR Priority: Medium #205 opened 2 weeks ago by OliverTchum • Changes requested 11 of 14 tasks Final Release				4			23
☐	Add tests to cover dashboard ✗ enhancement frontend PR Priority: Medium testing #183 opened last month by Dyno8K • Review required 10 of 14 tasks Final Release				10			
☐	Image upload backend ✗ backend enhancement PR Priority: High #150 opened on Feb 15 by AravRaja • Review required 9 of 14 tasks Final Release							

Thorough,
in-depth reviews

PR labels/metadata
for organisation and
planning

☐	6 Open ✓ 43 Closed	Author ▾	Label ▾	Projects ▾	Milestones ▾	Reviews ▾	Assignee ▾	Sort ▾
☐	Database ✓ backend enhancement PR Priority: Medium #106 by jack-jarvis12 was merged on Feb 11 • Approved 10 of 14 tasks Beta Release					8		50
☐	Google sign-in/sign-up implementation ✓ backend enhancement frontend PR Priority: Medium #177 by OliverTchum was merged 3 weeks ago • Approved 11 of 14 tasks Final Release				2			43
☐	Backend Live Chat Functionality ✓ backend enhancement PR Priority: Medium #170 by KacperZdrada was merged 3 weeks ago • Approved 10 of 14 tasks Final Release				5			37
☐	Dashboard frontend ✓ frontend PR Priority: High #153 by AravRaja was merged on Feb 21 • Approved 10 of 14 tasks Beta Release				5			31
☐	Dashboard improvements ✓ backend frontend PR Priority: High testing #207 by jack-jarvis12 was merged last week • Approved 10 of 14 tasks Final Release				8			31

Agile Development / Project Management

Issue labels/metadata for organisation and planning

[Bug] - Failure to make account from invalid housing association #190

Closed Bug #176 spe-uob/2024-SocialHousingTenantPortal Private

KacperZdrada opened on Mar 10

Bug Details

The bug occurs when trying to sign up to the service, but providing some invalid information, like an invalid housing association.

Actual Behaviour:
When an invalid housing association is sent as part of the request to `api/authentication/make_tenant/`, the following occurs:

- The server responds to the request with a `500` error.
- However, when trying to sign up again with the same email, the following error occurs: `"Conflict: Account already exists", status=409`.
- As such, it would make sense to attempt to login using those account details. However, again an error occurs: `"Incorrect email or password", status=401`.

Thus, it is now impossible to sign-up or login with said email.

Expected Behaviour:
If an error occurs during sign-up, no account should be created, and the email should still be free for use for sign-up.

Bug Explanation:
This bug occurs due to a minor logic error in the code for creating a tenant account, which then causes all of the rest of the errors too. All code referenced is within the following filepath: `SHTP-backend/authentication/views.py`

In the `make_tenant` endpoint, the following code can be found:

```
# Checks if account already exists
if User.objects.filter(username=email):
    return JsonResponse("Conflict: Account already exists", status=409)
# Create new user and Tenant objects
user = User.objects.create_user(username=email, password=password)
body["tenantId"] = user.id
tenant = Tenant.fromJSON(body)
tenant.save()
```

Assignees
jack-jarvis12

Labels
backend bug

Type
Bug

Projects
SHTP Main Kanban

Priority
Priority: High

Size
Size: Extra Small

Estimate
Enter number...

Iteration
Choose an iteration

Start date
No date

End date
No date

Milestone
Final Release

Relationships
None yet

Development

Low priority/long term 1

This item hasn't been started

2024-SocialHousingTenantPortal #53
[Feature] - Add Retry Mechanism for OpenAI API
Priority: Medium Size: Medium
Final Release backend enhancement

Todo 3 Estimate: 0

This is ready to be picked up

2024-SocialHousingTenantPortal #190
[Bug] - Failure to make account from invalid housing association
Priority: High Size: Extra Small
Final Release backend bug

2024-SocialHousingTenantPortal #194
[Feature] - Add data management views
Priority: Medium Size: Medium
Final Release enhancement

2024-SocialHousingTenantPortal #196
[Feature] - Django emails
Priority: Medium Size: Small
Final Release enhancement

In progress 5 Estimate: 0

This is actively being worked on

2024-SocialHousingTenantPortal #175
[Feature] - Improve authorization
Priority: Medium Size: Small
Final Release backend enhancement

2024-SocialHousingTenantPortal #182
[Feature] - Add tests to cover the Dashboard
Priority: Medium Size: Large
Final Release enhancement frontend testing

2024-SocialHousingTenantPortal #184
[Feature] - Add tests to ensure dashboard renders correctly
Priority: Medium Size: Medium
Final Release enhancement frontend testing

Detailed issue descriptions following issue templates

Problem - Database



Initial database implementation:

- We initially decided to use MongoDB for our database and the **mongoengine** python library as an **Object Document Mapper**. This solution would provide benefits of both SQL and document based databases.

Problems:

- When we started implementing accounts we found other Django services like authentication and session management rely heavily on the Django ORM.
- We researched a few alternatives to fully integrate Django with MongoDB but they were all out of date or unreliable.
- We hacked together our own session management system but were not confident it was robust enough

Solution - Database



Switching databases:

- We switched our database to a PostgreSQL database hosted on AWS.
- We decided the simplicity and stability of using the Django ORM rather than writing custom code to link Django to MongoDB outweighed the benefits gained by using a document based database.
- This required significant reworking of code

Increased complexity:

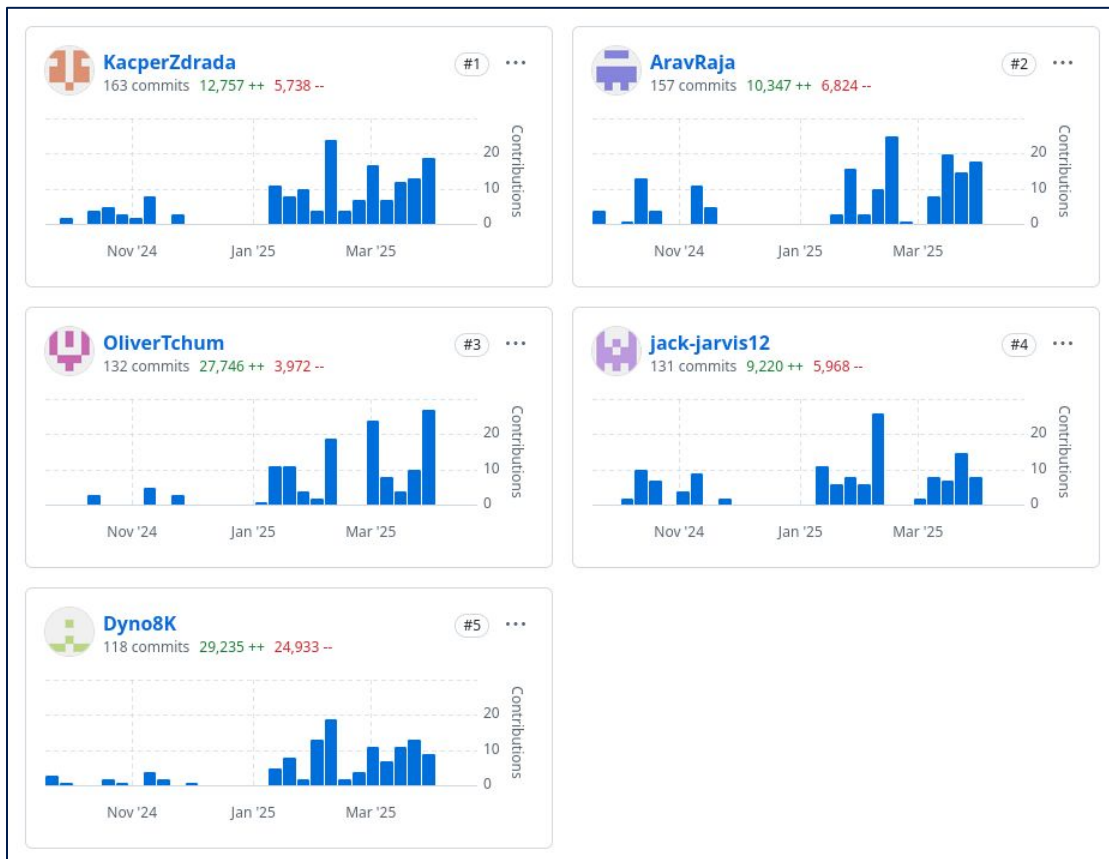
- Instead of saving files directly to the database, we are now using an AWS S3 storage bucket.
- To link our custom user data (addresses, contact details, etc) to the inbuilt Django user system, we have custom Tenant and Admin models which have one-to-one relationships with the Django User model.

Problem - Equal Workload

- In TB1 our **workload distribution** was raised as an issue
- Some team members were working more than others
- We were committing **too much code at once**

Solution - Equal Workload

- We performed a **team evaluation** at the beginning of TB2
- Started making **smaller** and **more regular** commits
- Introduced **rotating Project Manager** to alleviate workload on team members
- Began using formal **sprints**



Problem - Client Communication

- The main individual behind the project would be unable to communicate after December due to personal reasons
- All communication would therefore be with the technical lead of the project
- Unfortunately, this individual was much less available for meetings and progress updates due to time limitations
- Meetings were often rescheduled or cancelled at the last minute

Solution - Client Communication

Tenant Identification and Contextual Awareness

- ☐ The name information for login
- ☐ The housing association

Emergency

- ☐ The gas
- ☐ It must be emergency
- ☐ The

Issue Diagnosis

- ☐ The issue problem
- ☐ The more

Image/Video

- ☐ The auto issue
- ☐ It should be associated
- ☐ The meta

Responsibility Assessment (Tenant vs. Housing Association)

- ☐ The system must assess whether the issue falls under tenant responsibility (e.g., simple maintenance tasks, white goods, general upkeep) or requires intervention by the housing association.
- ☐ A decision tree or logic model must be implemented to guide this determination based on the issue's nature and housing association policies.

Tenant Fix Guidance

- ☐ If the system determines the issue can be fixed by the tenant, it should provide step-by-step instructions, including visual aids or video tutorials where appropriate.
- ☐ The system must ensure instructions are accessible and easy to understand, offering alternative formats if needed (e.g., text, video).

Work Order Generation and Dispatch

- ☐ If the issue is the responsibility of the housing association, the system must summarise the tenant's report into a structured work order form. This form should include:
 - ☐ Issue description
 - ☐ Relevant images/videos
 - ☐ Diagnostic information
 - ☐ Tenant's contact information and location
- ☐ The system must send this work order directly to the appropriate housing officer or repairs team via email, API integration, or other communication protocols.

Conversation Logging and Database Updates

- ☐ The system must save all tenant conversations, including text, media, and diagnostic data, to ensure a comprehensive record of the interaction.
- ☐ The tenant should be able to access their own conversation history
- ☐ It should update the tenant's record in the Housing Association database.
- ☐ The system must be able to synchronise with external APIs to integrate with the housing association's existing systems, ensuring seamless updates and communication between different departments.

System override

- ☐ Where the AI is having issues understanding/troubleshooting or if the tenant is becoming frustrated, we should be able to override the system and a human (from our team) should be able to enter the conversation - at least during the initial testing/pilot phases.

- We ensured to have as many **meetings with main client** as possible before January
- We performed **robust requirements gathering** from her for the entire scope of the project
- This allowed us to continue with **good progress** through TB2 even with much more limited client communication

Problem - Unrealistic Client Expectations

- Our client initially had very **unrealistic expectations** as to what we could achieve by the end of the year
- They wanted a **universal plugin tool** for existing portals, which would pose large **integration difficulties** due to varying APIs and codebases
- They wanted a **proprietary computer vision model** but were unable to provide any training data. After speaking with computer vision experts at the university, we decided this was not feasible.

Solution - Unrealistic Client Expectations

Google Login

- Instead of a universal plugin tool, we integrated Google's **“one-click” login** onto our standalone website
- This would allow for the best ease-of-access for tenants

OpenAI Computer Vision Functionality

- Instead of creating our own models, we would utilise **OpenAI's existing computer vision** functionality, seamlessly integrating with the LLM technology we were already using

Please enter your details

Welcome back!

Tenant Admin

Enter your email

Enter your password

[Forgot Password?](#)

Log in

 Sign in with Google

Don't have an account? [Sign Up](#)

What we would have done differently

- Used more **AGILE processes**, like sprints, from the very beginning of the project
- Utilised a **Gantt Chart** for easier timeline planning rather than just the **Kanban Board**
- Written React code with **TypeScript** instead of JavaScript for type safety
- Written all backend code with **WebSockets** rather than having a mix of **HTTP Polling** and **WebSockets**

Thank you!

<https://socialhousingtenantportal.live/>